

MicroDuck su ArduPilot

Come sono collegati MAVProxy, ArduRover, la rete PPO e MuJoCo

Roberto Navoni — DelphyAI LAB · r.navoni74@gmail.com · 24 settembre 2026

Repo: github.com/virtualrobotix/microduck-ap-ppo-sitl · fork ArduPilot: [virtualrobotix/ardupilot @ microduck-ppo](#)

Un solo firmware ArduRover. La rete neurale (61 osservazioni → 14 servo, 50 Hz) è un task dello scheduler. MuJoCo è il corpo del robot, collegato al SITL come un motore fisico esterno. Il comando arriva da MAVProxy via MAVLink.

1. Tre processi, due collegamenti

In simulazione: GCS, firmware, corpo del robot. Su hardware il terzo processo è il robot vero.

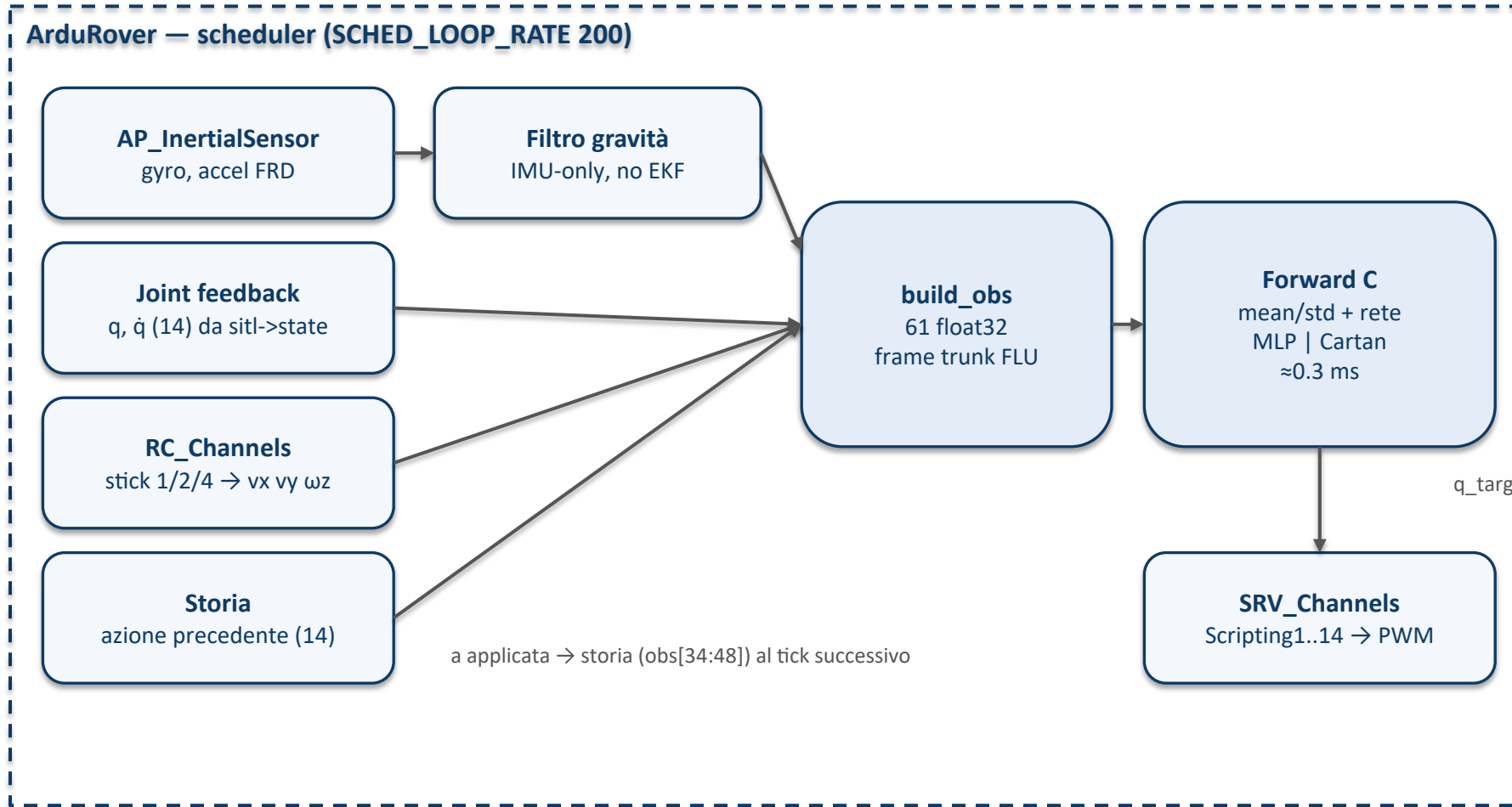


Lock-step: il SITL invia i PWM e aspetta il JSON; MuJoCo fa un passo da 5 ms per frame (SIM_RATE_HZ 200). Il task PPO gira ogni 4 frame = 50 Hz, come in training.

Il task PPO non ha alcun collegamento con MuJoCo: vede solo API ArduPilot. Sostituire la pianta con IMU + bus Dynamixel reali non cambia una riga di AP_MicroDuck.

2. Dentro ArduRover: il loop a 50 Hz di AP_MicroDuck

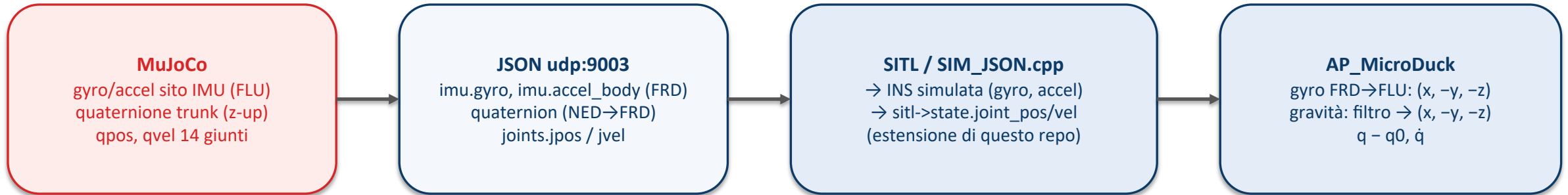
Tutto ciò che la rete vede passa dalle librerie standard di ArduPilot.



- `update_attitude()` a loop rate: propaga la direzione della gravità col gyro, la corregge con l'accelerometro (MDK_ATT_TAU 0.5 s)
- `update()` a 50 Hz: obs → forward → clip → PWM
- Failsafe: disarmato, feedback giunti assente o vecchio > MDK_WD_MS → azione 0 = posa stand (anche nella storia)
- HOLD (MDK_HOLD_MODE 4) → twist 0
- PWM: $1500 + q/0.003$ ($1 \mu s = 3 \text{ mrad}$), SERVOn 800..2200

3. Percorso dei sensori: da MuJoCo all'osservazione

Frame diversi, unità diverse: la pianta parla ArduPilot (FRD/NED), il task parla training (FLU).



Conversione dei frame (fatta dalla pianta)

- vettori: $(x, y, z)_{\text{MuJoCo}} \rightarrow (x, -y, -z)_{\text{ArduPilot}}$ — vale per gyro, accel, posizione, velocità
- quaternion: $(w, x, y, z) \rightarrow (w, x, -y, -z)$
- in piedi: $\text{accel_body} = (0, 0, -9.81)$ come ArduPilot si aspetta; gravità proiettata nel task = $(0, 0, -1)$ come in training

Lezione del primo test

- ArduRover ha INS_GYRO_FILTER 4 Hz di default: il gyro arrivava alla rete con decine di ms di ritardo → caduta in 1 s
- INS_GYRO_FILTER 0 (o 40) → stabile. Nessun passa-basso lento sul gyro che alimenta la policy, anche su hardware

4. Percorso dei comandi: da MAVProxy al twist

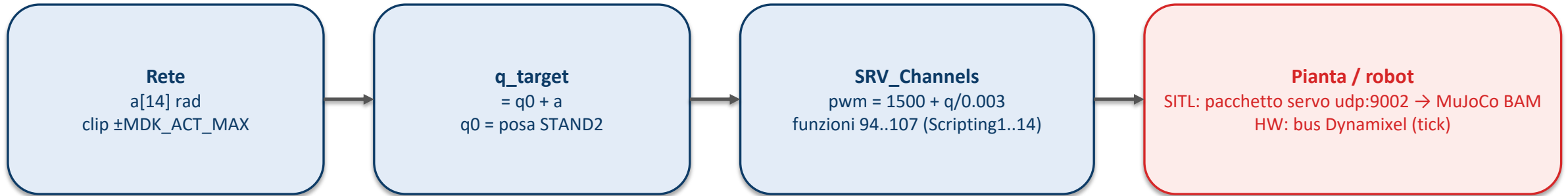
Ogni comando digitato in MAVProxy è un messaggio MAVLink standard; ArduPilot non è stato modificato qui.

Comando MAVProxy	Messaggio MAVLink	Cosa fa in ArduRover	Effetto sulla rete
arm throttle [force]	COMMAND_LONG MAV_CMD_COMPONENT_ARM_DISARM p1=1	AP_Arming → soft armed	PPO_FAIL 1→0, servo attivi, la policy tiene in piedi
rc 2 2000	RC_CHANNELS_OVERRIDE chan2=2000 (da sysid 255)	RC_Channels: norm_input_dz = +1.0	obs[48] vx = +1.0 × MDK_VX_MAX = 0.4 m/s
rc 1 1800	RC_CHANNELS_OVERRIDE chan1=1800	norm = +0.6	obs[49] vy = 0.6 × 0.3 = 0.18 m/s
rc 4 2000	RC_CHANNELS_OVERRIDE chan4=2000	norm = +1.0	obs[50] ωz = 1.0 rad/s
mode hold	COMMAND_LONG MAV_CMD_DO_SET_MODE custom_mode=4	Rover in HOLD	MDK_HOLD_MODE → twist forzato a 0: sta fermo
param set MDK_POLICY 1	PARAM_SET	AP_Param	prossimo tick usa la rete Cartan
disarm	COMMAND_LONG ARM_DISARM p1=0	soft disarmed	azione 0, storia azzerata, PWM 0 (pianta: idle)

Telemetria di ritorno (NAMED_VALUE_FLOAT ogni 0.5 s): PPO_MS tempo del forward, PPO_PGZ gravità z (in piedi ≈ −1), PPO_VX comando, PPO_FAIL stato.
Log DataFlash MDK / MDKQ / MDKV / MDKA: ogni tick con le 61 osservazioni e le 14 azioni → tools/log_parity.py le rigioca nell'ONNX (parità 2.5e-7).

5. Percorso degli attuatori: dall'azione ai servo

La rete produce offset in radianti; ArduPilot li trasporta come PWM; la pianta li riconverte in target di posizione.



- Perché Scripting1..14: sono funzioni servo già esistenti che ArduPilot non tocca (nessun mixer, nessun ESC), scrivibili con `SRV_Channels::set_output_pwm` senza modificare `SRV_Channel`
- Risoluzione 3 mrad/μs: sufficiente per gli XL330 (1.5 mrad/tick); su hardware si passa ai tick Robotis e la conversione sparisce
- PWM 0 = 'non guido' (disarmato): la pianta tiene il duck fermo in piedi; su hardware corrisponde a torque off
- L'azione che finisce nei servo è la stessa salvata nella storia: la rete al passo dopo vede ciò che è stato davvero applicato
- Feedback: la pianta restituisce q e $\dot{q}$ reali dei giunti; il task non usa mai i target come surrogato

6. L'integrazione in ArduRover, file per file

Fork virtualrobotix/ardupilot, branch microduck-ppo, sopra master upstream del 24/09/2026. 7 file toccati + 1 libreria nuova.

File	Modifica	Perché
libraries/AP_MicroDuck/ (nuova)	AP_MicroDuck.{h,cpp}, microduck_infer.{h,c}, policy_mlp.h, policy_cartan.h, AP_MicroDuck_config.h	il task: obs, filtro gravità, storia, stick, servo, MDK_*, log, telemetria; rete in C
Rover/Parameters.h / .cpp	membro g2.microduck; AP_SUBGROUPINFO "MDK_" indice 63	parametri MDK_* visibili da GCS e salvati in EEPROM
Rover/Rover.cpp	SCHED_TASK_CLASS update_attitude 400 Hz (→ loop rate), update 50 Hz	il task entra nello scheduler come qualunque libreria
Rover/wscript	+ 'AP_MicroDuck'	link della libreria nel binario ardurover
libraries/SITL/SIM_JSON.h / .cpp	tipo DATA_FLOAT_ARRAY14; chiavi joints/jpos, joints/jvel; copia in sitl->state	il backend fisico consegna i giunti come consegna l'IMU
libraries/SITL/SITL.h	sitl_fdm: joint_pos[16], joint_vel[16], joint_count, joint_time_us	sorgente SITL del joint feedback; su HW la sostituisce un driver Dynamixel

Nessuna modifica a RC_Channels, SRV_Channel, AP_InertialSensor, AP_Arming, GCS_MAVLink: il task usa le loro API pubbliche. AP_MICRODUCK_ENABLED è 1 su SITL; le board devono abilitarlo (pesi ~770 KB + ~500 KB di flash in float32).

Parametri SITL (sitl/microduck.parm): SERVO1..14_FUNCTION 94..107, SERVOn_MIN/MAX 800/2200, SIM_RATE_HZ 200, SCHED_LOOP_RATE 200, INS_GYRO_FILTER 0, ARMING_CHECK 0, MDK_ENABLE 1.

7. Tempi e sincronizzazione

Una sola linea del tempo, guidata dal lock-step JSON.

MuJoCo (5 ms)



SITL frame / INS (5 ms)



Loop ArduRover (5 ms)



AP_MicroDuck::update (20 ms)



MAVProxy (asincrono)

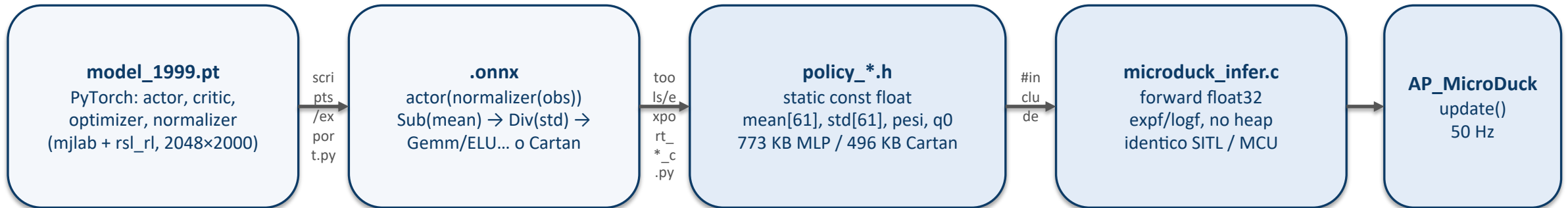


200 ms

- SITL manda i PWM → MuJoCo fa 1 passo (5 ms) → risponde col JSON → il SITL avanza il suo clock di 5 ms. Nessuno dei due può correre avanti.
- Il task a 50 Hz vede 4 passi fisici per azione, come in training (decimation 4). Il forward dura 0.25–0.37 ms sul Mac; su STM32H7 stimati 1–2 ms.

8. Da .pt a C: come la rete addestrata entra nel firmware

Il normalizzatore delle osservazioni è parte della rete, non del simulatore.



Verifica in tre punti

- `tools/parity_check.py`: 2201 osservazioni → ONNX Runtime vs binario C: max 3.8e-6 (MLP), 6.0e-5 (Cartan)
- `tools/log_parity.py`: le osservazioni loggate dal firmware in volo → ONNX vs le azioni che il firmware ha inviato: max 2.5e-7
- `tools/policy_rollout_plant.py`: la stessa policy sulla stessa pianta senza ArduPilot — separa problemi di pianta da problemi di firmware

Le due reti (MDK_POLICY)

- 0 = MLP 61→512→256→128→14 ELU, 197 896 parametri, 0.37 ms
- 1 = Cartan+DiLU 61→192→3×CartanLinear→14, 127 054 parametri (-36 %), 0.25 ms, operatore con exp/log in C

9. Cosa cambia passando all'hardware

Solo il terzo processo: la pianta MuJoCo diventa il robot. AP_MicroDuck non cambia.

Blocco	In SITL (oggi)	Su robot
IMU	MuJoCo → JSON → INS simulata	IMU del flight controller (stesso AP_InertialSensor)
Feedback giunti	JSON joints → sitl->state	driver bus Dynamixel (present position/velocity) → set_joint_feedback()
Uscite servo	SRV_Channels → pacchetto JSON PWM → MuJoCo BAM	SRV_Channels → protocollo Robotis (tick) sugli XL330
Comandi	MAVProxy rc / mode / arm	RC reale + GCS, identici
Rete	policy_*.h in C, 0.3 ms su Mac	stessa sorgente; STM32H7: 1–2 ms stimati (MLP), Cartan da misurare
Filtri IMU	INS_GYRO_FILTER 0	stessa regola: nessun LPF lento sul gyro che alimenta la policy

Il vincolo vero è il budget del microcontrollore: $\sim 2 \times 10^5$ MAC per passo (MLP) a 50 Hz più EKF, bus e logging. Il prossimo passo è lo stesso update() compilato su una Nucleo H7 con 1000 forward e misura p50/p99.

10. La demo e i risultati

scripts/demo_mavproxy.py: un comando avvia pianta, SITL e MAVProxy e digita la sequenza dell'operatore.

Sequenza digitata in MAVProxy

- param set MDK_POLICY 0|1 · param set MDK_ENABLE 1
- mode manual · rc all 1500 · arm throttle → 6 s in piedi
- rc 2 2000 → avanti 3 s · rc 2 1000 → indietro 2 s
- rc 1 1800 → laterale 2 s · rc 4 2000 → rotazione 90° (integrale di ATTITUDE.yawspeed)
- rc 2 2000 → avanti 5 s
- mode hold → twist 0 · mode manual · disarm

Nel video (docs/media/demo_mavproxy_mlp.mp4)

- box crediti e pannello a sinistra: comando MAVProxy e messaggio MAVLink corrispondente
- badge a destra: fase e twist comandato in SI
- barra in basso: tempo, posizione, tilt del robot in MuJoCo

	MLP	Cartan
Parità C vs ONNX	3.8e-6	6.0e-5
Parità in-situ (log firmware)	2.5e-7	1.3e-6 (p99)
Forward nel SITL	0.37 ms	0.25 ms
Flash pesi (float32)	773 KB	496 KB
Batteria HIL 8 passi	8/8 PASS	8/8 PASS
Avanti 0.3/0.4 m/s 15 s	nessuna caduta	—

Il tracking di velocità (~0.2 m/s a comando 0.4) è lo stesso del rollout Python sulla stessa pianta CPU: è il gap MuJoCo-CPU vs MuJoCo-Warp del training, non l'integrazione ArduPilot.